



Institute of Engineering and Technology (IET)

Minor Project (PR1103)

End-Term Report

**Academic Performance Evaluator &
Learning Management System (LMS)**

PREPARED BY

Sarthak Pandit (2023BTECH072)

Tushar Parihar (2023BTECH093)

FACULTY GUIDE

Mr. Santosh Kumar Verma
Computer Science and Engineering

Mr. Gaurav Raj
Computer Science and Engineering

Academic Year 2025-2026

Abstract

This report documents the complete design, implementation, and evaluation of an **Academic Performance Evaluator and Learning Management System (LMS)** developed as a B.Tech Minor Project. The system is a full-stack web application built with **React 18** on the frontend and a **Node.js / Express / MongoDB** backend, providing role-based academic workflows for Administrators, Teachers, and Students.

The platform covers quiz and assignment management, study materials, notification systems, performance bucketing, and – most notably – an **AI-powered Performance Insights engine** embedded directly in the Student Dashboard. The AI module leverages the **Google Gemini API** to analyse each student's real quiz results and assignment submission data and produce a structured performance report covering overall standing, identified strengths, weak areas, actionable improvement suggestions, and study tips.

The system exposes **40+ REST API endpoints** across nine API groups and maintains a clean, extensible data model with 14 MongoDB collections. Attendance tracking and timetable management are identified as near-term extensions and are documented in the future roadmap. This report presents the full architecture, module descriptions, API catalogue, database schema, folder structure, current limitations, and future roadmap.

Contents

Abstract	1
1 Introduction	4
1.1 Background and Motivation	4
1.2 Naive Approach and Why We Moved Beyond It	4
1.3 Scope of This Report	4
2 Objectives	5
3 Technology Stack	6
3.1 Frontend Technologies	6
3.2 Backend Technologies	6
3.3 Database & Infrastructure	6
3.4 AI Integration: Google Gemini	7
4 System Architecture	8
4.1 High-Level Layered Design	8
4.2 Role-Based Routing	8
4.3 Backend Route Mounting	9
5 Project Folder Structure	10
5.1 Root Layout.....	10
5.2 Frontend (client/).....	10
5.3 Backend (server/).....	11
5.4 Key File Responsibilities.....	12
6 Feature Modules	13
6.1 Core Identity and Access Management	13
6.2 Admin Module.....	13
6.3 Course Management	13
6.4 Teacher Module	13
6.5 Quiz and Assessment Module	13
6.6 Assignment Module.....	14
6.7 Study Materials Module	14
6.8 Notification System	14
6.9 Performance Bucketing and Promotion.....	14
7 AI Performance Insights Module	15
7.1 Design Decision	15
7.2 UI Changes.....	15
7.3 AILearningAssistant Component Workflow.....	15
7.4 Server-Side Gemini Service	16
7.5 AI Output Structure.....	16
7.6 Modal Sections	17
7.7 Prompt Engineering Principles.....	17
7.8 Error Handling.....	17
8 API Catalogue	18
8.1 Authentication.....	18

8.2	Admin	18
8.3	Courses	18
8.4	Quizzes	19
8.5	Assignments.....	19
8.6	Materials.....	19
8.7	Notifications	19
8.8	Buckets	20
9	Database Schema	21
9.1	Identity Collections	21
9.2	Academic Core Collections	21
9.3	Operations Collections.....	21
9.4	Progression Collections	22
10	Frontend Component Architecture	23
10.1	Role-Based Layout Components.....	23
10.2	Student-Facing Components	23
10.3	Key Teacher Components	23
11	Security and Data Integrity	24
11.1	Authentication and Authorisation	24
11.2	Data Integrity Mechanisms	24
11.3	AI Safety and Fairness	24
12	Current Strengths and Limitations	25
12.1	Strengths	25
12.2	Limitations and Improvement Opportunities	25
13	Future Scope	26
13.1	Short-Term – Planned Modules.....	26
13.2	Medium-Term	26
13.3	Long-Term.....	26
14	Risk Register	27
15	Conclusion	28
16	References	28
	Appendix A – Data Model Quick Reference	29
	Appendix B – Environment Variables	29
	Appendix C – Setup & Run Instructions	29

1. Introduction

1.1 Background and Motivation

Modern educational institutions face growing challenges in tracking student performance across multiple subjects, assessment types, and learning stages. Traditional LMS platforms often treat assessments in isolation and lack intelligent feedback loops that connect performance data to personalised guidance.

This project addresses that gap by building a full-featured academic platform that manages the operational aspects of teaching and learning (enrolment, materials, assessments) and simultaneously applies **AI-driven analysis** to transform raw academic data into actionable student insights.

1.2 Naive Approach and Why We Moved Beyond It

The simplest possible implementation of a performance evaluator would be a spreadsheet or a static dashboard: a teacher manually enters marks, a formula computes a percentage, and a pass/fail label is displayed. While trivially easy to build, this approach suffers from several fundamental shortcomings.

- **No role separation.** A single table of marks gives no distinction between what a student, teacher, or administrator should see or do.
- **No feedback loop.** A raw percentage tells a student they scored 62% but gives no guidance on *why* or *what to do next*.
- **No adaptive difficulty.** Every student receives the same content regardless of mastery level.
- **No automation.** Question creation, feedback generation, and recommendation surfacing all require manual teacher effort.
- **No scalability.** A spreadsheet breaks down as student count, course count, or assessment frequency grows.

Our system replaces this naive baseline with a structured three-role REST API platform, an AI-powered insights layer (Google Gemini), and a bucket-based adaptive difficulty tracker – each directly addressing one of the above shortcomings.

1.3 Scope of This Report

This report covers the complete project state including all implemented features, the AI Performance Insights module (powered by Google Gemini), the API catalogue, schema documentation, architectural decisions, complete folder structure, and the future roadmap for planned modules.

2. Objectives

1. Build a complete academic lifecycle platform covering all core institutional operations for three distinct user roles.
2. Strengthen evaluation mechanisms through quizzes, assignments, and multi-source analytics.
3. Provide robust, role-specific REST APIs for Admin, Teacher, and Student dashboards.
4. Implement AI-powered student performance analysis using the **Google Gemini API** to deliver personalised, data-grounded insights.
5. Introduce bucket-based adaptive learning tracking to identify each student's current difficulty stage per subject.
6. Keep the system extensible for future modules (attendance, timetable) and long-term goals (mobile clients, multi-institution tenancy).

3. Technology Stack

3.1 Frontend Technologies

Technology	Purpose
React 18	Component-based UI framework for all three role dashboards
Vite	Fast development server and optimised production build toolchain
React Router v6	Client-side routing and nested layout management
Tailwind CSS	Utility-first styling for a consistent design language
Recharts	Line charts, radar charts, and bar charts for analytics
Lucide React	SVG icon library (Brain, BookOpen, Bell, Target, etc.)
Radix Primitives	Accessible headless UI primitives

Table 1: Frontend Technology Stack

3.2 Backend Technologies

Technology	Purpose
Node.js 20+	JavaScript runtime for the REST API server
Express 4	Web framework for route mounting and middleware
Mongoose 7	ODM for MongoDB with schema validation
JWT	Stateless authentication and route protection
Multer	Multipart file upload handling (PDF, images, etc.)
pdf-parse	Server-side PDF text extraction for AI workflows
Google Gemini SDK	@google/generative-ai – integration with gemini-pro

Table 2: Backend Technology Stack

3.3 Database & Infrastructure

Component	Details
MongoDB Atlas / Local	Document-oriented database; 14 collections
Mongoose Schemas	Role-separated data models with field-level validation
Indexes	Unique compound indexes prevent duplicate records

Table 3: Database Stack

3.4 AI Integration: Google Gemini

The project uses the **Google Gemini API** (gemini-pro model) for all AI-driven features. Gemini was chosen because it offers a generous free-tier quota suitable for academic prototypes, strong support for structured JSON output via prompt engineering, a native Node.js SDK (@google/generative-ai) for seamless back-end integration, and a sufficient context window to handle multi-subject student data payloads in a single request. The API key is stored server-side in a .env file (GEMINI_API_KEY) and is *never* exposed to the frontend client.

4. System Architecture

4.1 High-Level Layered Design

The system follows a classic three-tier architecture:

1. **Presentation Layer** – React-based SPA with role-specific layouts (StudentLayout, TeacherLayout, AdminLayout) and protected route guards.
2. **Application Layer** – Express REST API with modular route / controller / model separation across nine API groups.
3. **Data Layer** – MongoDB collections, with Mongoose schemas enforcing constraints and relationships.

Presentation Layer – React 18 SPA (Student / Teacher / Admin)



Application Layer – Express REST API (9 route groups)



Data Layer – MongoDB (14 Collections via Mongoose)

4.2 Role-Based Routing

The `createRouterForRole()` function dynamically constructs the React Router configuration based on the authenticated user's role, ensuring each role sees only the routes relevant to their access level.

```
export function createRouterForRole (role) {
  if (role === 'student') {
    return createBrowserRouter ([
      { path: "/", Component: ProtectedStudentLayout, children: [
        { index: true, Component: StudentPortal },
        { path: "courses", Component: Courses },
        { path: "assignments", Component: Assignments },
        { path: "materials", Component: StudyMaterials },
        { path: "quizzes", Component: StudentQuizList },
        { path: "quiz/:id", Component: StudentQuizTake },
        { path: "my-progress", Component: StudentBucketProgress },
      ]},
    ])
  }
  // teacher and admin branches follow the same pattern
}
```

Listing 1: Role-based router creation (routes.tsx)

4.3 Backend Route Mounting

Mount Point	Responsibility
/api/auth	Registration and login
/api/admin	Institution management
/api/courses	Course CRUD and enrolment
/api/teachers	Teacher profile and marks
/api/quizzes	Quiz lifecycle and submissions
/api/assignments	Assignment lifecycle and AI helpers
/api/materials	File/link resource management
/api/notifications	Role-targeted notifications
/api/buckets	Bucket tracking and promotion

Table 4: API Route Groups (attendance and timetable planned for next release)

5. Project Folder Structure

The repository is a monorepo with two workspaces: client/ (React frontend) and server/ (Node.js backend).

5.1 Root Layout

```
LMS - System - Minor - Project/
|-- client/           # React 18 + Vite frontend
|-- server/           # Node.js + Express backend
|-- .gitignore
|-- README.md
```

Listing 2: Top-level repository structure

5.2 Frontend (client/)

```
client/
|-- public/
|   |-- favicon.ico
|-- src/
|   |-- components/
|   |   |-- layouts/
|   |   |   |-- StudentLayout.tsx
|   |   |   |-- TeacherLayout.tsx
|   |   |   |-- AdminLayout.tsx
|   |   |-- admin /
|   |   |   |-- StudentApprovals.tsx
|   |   |   |-- TeacherApprovals.tsx
|   |   |   |-- AdminDashboard.tsx
|   |   |   |-- CourseManagement.tsx
|   |   |-- teacher/
|   |   |   |-- TeacherDashboard.tsx
|   |   |   |-- SubjectMarks.tsx
|   |   |   |-- BucketDashboard.tsx
|   |   |   |-- TeacherQuizManagement.tsx
|   |   |   |-- TeacherQuizMonitor.tsx
|   |   |   |-- TeacherNotifications.tsx
|   |   |   |-- AssignmentManagement.tsx
|   |   |-- student/
|   |   |   |-- AllLearningAssistant.tsx
|   |   |   |-- StudentQuizList.tsx
|   |   |   |-- StudentQuizTake.tsx
|   |   |   |-- StudentBucketProgress.tsx
|   |   |   |-- Assignments.tsx
|   |   |   |-- StudyMaterials.tsx
|   |   |   |-- Courses.tsx
|   |   |-- shared /
|   |   |   |-- ProtectedRoute.tsx
|   |   |   |-- ProtectedTeacherRoute.tsx
|   |   |   |-- Spinner.tsx
|   |   |   |-- StudentPortal.tsx
|   |-- contexts/
|   |   |-- AuthContext.tsx
|   |-- hooks/
```

```
| | | -- useAuth.ts  
| | | -- useFetch.ts  
| | -- utils/  
| | | -- api.ts  
| | | -- helpers.ts  
| | -- routes.tsx  
| | -- App.tsx  
| | -- main.tsx  
| | -- index.css  
|-- index.html  
|-- vite.config.ts  
|-- tailwind.config.js  
|-- tsconfig.json  
|-- package.json
```

Listing 3: Frontend folder structure

5.3 Backend (server/)

```
server/  
|-- config /  
|   |-- db.js  
|-- controllers/  
|   |-- authController.js  
|   |-- adminController.js  
|   |-- courseController.js  
|   |-- teacherController.js  
|   |-- quizController.js  
|   |-- assignmentController.js  
|   |-- materialController.js  
|   |-- notificationController.js  
|   |-- bucketController.js  
|-- middleware /  
|   |-- authMiddleware.js  
|   |-- uploadMiddleware.js  
|   |-- errorMiddleware.js  
|-- models/  
|   |-- Student.js  
|   |-- Teacher.js  
|   |-- Admin.js  
|   |-- User.js  
|   |-- Course.js  
|   |-- Quiz.js  
|   |-- QuizResult.js  
|   |-- Assignment.js  
|   |-- AssignmentSubmission.js  
|   |-- Material.js  
|   |-- Notification.js  
|   |-- SubjectThreshold.js  
|   |-- StudentBucket.js  
|-- routes/  
|   |-- authRoutes.js  
|   |-- adminRoutes.js  
|   |-- courseRoutes.js  
|   |-- teacherRoutes.js  
|   |-- quizRoutes.js  
|   |-- assignmentRoutes.js  
|   |-- materialRoutes.js
```

```
|  |-- notificationRoutes.js
|  |-- bucketRoutes.js
|-- services/
|  |-- geminiService.js
|-- uploads/
|-- .env
|-- .env.example
|-- server.js
|-- package.json
```

Listing 4: Backend folder structure

5.4 Key File Responsibilities

Path	Responsibility
client/src/routes.tsx	Router factory for all three roles
client/src/components/layouts/StudentLayout.tsx	Student sidebar navigation
client/src/components/StudentPortal.tsx	Student home dashboard
client/src/components/student/AILearningAssistant.tsx	Gemini AI Insights modal
client/src/components/student/StudentQuizTake.tsx	Timed quiz-taking interface
client/src/components/student/StudentBucketProgress.tsx	Bucket progress view
client/src/components/teacher/BucketDashboard.tsx	Bucket management for teachers
client/src/contexts/AuthContext.tsx	Global authentication state
server/services/geminiService.js	Gemini API calls and prompt logic
server/controllers/quizController.js	Quiz CRUD + AI question generation
server/controllers/assignmentController.js	Assignment CRUD + AI helpers
server/models/StudentBucket.js	Bucket progression schema
server/.env	Secret keys (never committed to git)

Table 5: Key Source File Index

6. Feature Modules

6.1 Core Identity and Access Management

- Role-based registration and login for **Admin**, **Teacher**, and **Student** roles.
- **Approval workflow**: newly registered students and teachers enter a *pending* state and gain access only after Admin approval.
- JWT-based authentication with token verification middleware protecting all sensitive routes.
- ProtectedRoute and ProtectedTeacherRoute components enforce route-level guards on the frontend.

6.2 Admin Module

- View and act on pending/approved student and teacher registrations.
- Create courses, enrol students by semester/department criteria, and assign teachers to courses.
- Institutional overview via /api/admin/dashboard-stats.
- Student approvals managed through the StudentApprovals component.

6.3 Course Management

- Course search by name or code; semester-wise listing.
- Bulk enrolment by semester (enroll-semester endpoint).
- Teacher–course linkage with enrolled student population management.

6.4 Teacher Module

- Personal profile (/me) and assigned course list (/me/courses).
- Enriched marks view combining quiz scores and manual marks into a unified subject-level report.
- SubjectMarks component for per-course grade management.
- Notifications managed via TeacherNotifications.

6.5 Quiz and Assessment Module

- Full quiz lifecycle: create → publish → attempt → result.
- Support for difficulty levels and negative marking.
- **AI-assisted quiz question generation** via /api/quizzes/generate-ai, powered by Gemini.
- Per-student result retrieval and attempt history.

- TeacherQuizMonitor for live monitoring of ongoing sessions.
- Star-based rating (1–5 stars) based on percentage score shown on the Student Portal.

6.6 Assignment Module

- Create, update, publish, and delete assignments with multiple question modes (quiz mode / solve mode).
- Student submission workflow with teacher review and scoring.
- **AI helpers (all powered by Gemini):**
 - /generate-ai – automatic question generation from a topic.
 - /extract-pdf – extract questions from uploaded PDFs.
 - /gemini – primary Gemini generation endpoint.
 - /ai-performance – per-submission AI feedback.
 - /generate-variations – anti-plagiarism question variants.
- AI recommendations are persisted to localStorage and surfaced in the Student Portal's Recommendations section.

6.7 Study Materials Module

- File uploads supporting PDF, DOC/DOCX, PPT, images, and video.
- External link-based material entries.
- Filtered retrieval by course, subject, and student enrolment.
- Download tracking and soft-delete with ownership checks.

6.8 Notification System

- Role-based and user-targeted notifications.
- Mark individual or all notifications as read.
- Soft-delete via isActive flag (no destructive deletes).

6.9 Performance Bucketing and Promotion

- Per-course promotion thresholds configurable by the teacher.
- Students tracked in one of three levels per subject: **Easy** → **Medium** → **Hard**.
- Manual bucket override and promotion triggers available to faculty.
- Course-level overview showing class distribution across buckets.
- StudentBucketProgress provides a visual progression summary across all subjects.

7. AI Performance Insights Module

This is the primary new feature of this release. It replaces the previous standalone “AI Assistant” navigation item with a contextually embedded, data-driven insights panel accessible directly from the Student Portal home page. The AI backbone is the **Google Gemini API**.

7.1 Design Decision

The earlier prototype surfaced AI as a separate page (/ai-assistant) in the sidebar. Students rarely visited it because it felt disconnected from their live dashboard data. The redesign **removes the standalone route** and places a compact “**AI Insights**” button directly inside the student profile card, making the feature one click away from real performance data.

7.2 UI Changes

1. **Removed:** “AI Assistant” from StudentLayout sidebar.
2. **Removed:** ai-assistant route from routes.tsx.
3. **Added:** “AI Insights” button inside the student profile card on StudentPortal.tsx.
4. **Added:** Full-screen overlay modal with gradient header (violet → indigo), scrollable content, and close button.

```
<button
  onClick={() => setShowAIPanel(true)}
  title="AI Performance Insights"
  className="flex items-center gap-1.5 px-3 py-1.5
    bg-white/20 hover:bg-white/30 border border-white/30
    text-white rounded-lg text-xs font-medium transition-colors"
>
  <Brain className="w-3.5 h-3.5" />
  <span>AI Insights </span>
</button>
```

Listing 5: AI Insights button in StudentPortal.tsx

7.3 AILearningAssistant Component Workflow

The component receives `userId` and `userName` as props and executes four steps:

1. **Data collection** – Fetches enrolled courses, quiz results, and assignment submissions for the student.
2. **Summarisation** – Aggregates scores, grades, and submission rates into a compact JSON payload per subject.
3. **Gemini call** – POSTs the payload to `/api/assignments/gemini` on the backend, which forwards it to `gemini-pro` via `geminiService.js`.
4. **Rendering** – Parses the JSON response and displays five labelled sections inside the modal.

7.4 Server-Side Gemini Service

```
const { GoogleGenerativeAI } = require("@google/generative-ai");

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

async function analyseStudentPerformance(studentData) {
  const model = genAI.getGenerativeModel({ model: "gemini-pro" });

  const prompt = `You are an academic performance advisor.
  Analyse the student data below and return ONLY a valid JSON object
  with no markdown fencing.

  Student Data:
  ${JSON.stringify(studentData, null, 2)}

  Return exactly this JSON shape:
  {
    "overallStatus": "Excellent | Good | Needs Improvement",
    "summary": "2-3 sentence overview",
    "strengths": ["..."],
    "weakAreas": ["..."],
    "improvements": ["..."],
    "studyTips": ["..."]
  }`;

  const result = await model.generateContent(prompt);
  const text = result.response.text();
  return JSON.parse(text);
}

module.exports = { analyseStudentPerformance };
```

Listing 6: server/services/geminiService.js

7.5 AI Output Structure

```
{
  "overallStatus": "Good",
  "summary": "...",
  "strengths": ["Mathematics", "Data Structures"],
  "weakAreas": ["Operating Systems", "DBMS"],
  "improvements": ["Revise OS scheduling algorithms",
    "Practice SQL JOIN queries"],
  "studyTips": ["Use spaced repetition for theory",
    "Attempt previous year papers"]
}
```

Listing 7: Expected JSON response from Gemini

7.6 Modal Sections

Section	Content
Overall Status	Colour-coded badge: green (Excellent), yellow (Good), red (Needs Improvement)
Performance Summary	2–3 sentence overview of academic standing
Strengths	Bulleted list of subjects/skills the student excels in
Weak Areas	Bulleted list of subjects requiring focused attention
Areas of Improvement	Specific, actionable steps the student should take
Study Tips	General strategies (spaced repetition, practice tests)

Table 6: AI Insights Modal – Section Breakdown

7.7 Prompt Engineering Principles

- Gemini is instructed to act as an academic advisor, not a general assistant.
- Only provided data is used – no invented subject names or scores.
- The model must return *only* a valid JSON object; the backend calls `JSON.parse()` before forwarding to the frontend.
- Language is kept student-friendly, specific, and constructive.

7.8 Error Handling

- A loading spinner is shown while data is fetched and the Gemini call is in progress.
- If the student has too little activity, a graceful message prompts them to complete more quizzes and assignments.
- `JSON.parse()` failures are caught and display an error state rather than crashing the UI.

8. API Catalogue

8.1 Authentication

Endpoint	Description
POST /api/auth/register	Register student / teacher / admin
POST /api/auth/login	Authenticate and receive JWT

8.2 Admin

Endpoint	Description
GET /api/admin/students/pending	Pending registrations
GET /api/admin/students/approved	Approved students
GET /api/admin/students/:id	Student detail
POST /api/admin/students/:id/approve	Approve student
POST /api/admin/students/:id/reject	Reject student
GET /api/admin/teachers/pending	Pending teachers
GET /api/admin/teachers/approved	Approved teachers
POST /api/admin/teachers/:id/approve	Approve teacher
GET /api/admin/courses	List all courses
POST /api/admin/courses	Create course
GET /api/admin/courses/:id/students	Course enrolment list
POST /api/admin/courses/:id/enroll	Enrol student
PATCH /api/admin/courses/:id/assign-teacher	Assign teacher
GET /api/admin/dashboard-stats	Institutional statistics

8.3 Courses

Endpoint	Description
GET /api/courses/search?q=	Search by name/code
GET /api/courses/semester/:sem	By semester
POST /api/courses	Create course
POST /api/courses/:id/enroll	Enrol student
POST /api/courses/:id/enroll-semester	Bulk semester enrol

8.4 Quizzes

Endpoint	Description
POST /api/quizzes/generate-ai	AI question generation (Gemini)
POST /api/quizzes	Create quiz
PATCH /api/quizzes/:id/publish	Publish quiz
GET /api/quizzes/course/:courseId	Quizzes by course
POST /api/quizzes/:id/submit	Submit attempt
GET /api/quizzes/:id/attempts	All attempts
GET /api/quizzes/:id/result/:studentId	Student result

8.5 Assignments

Endpoint	Description
POST /api/assignments/generate-ai	AI question generation (Gemini)
POST /api/assignments/extract-pdf	Extract questions from PDF
POST /api/assignments/gemini	Primary Gemini AI endpoint
POST /api/assignments/ai-performance	AI submission feedback
POST /api/assignments/generate-variations	Question anti-plagiarism variants
PATCH /api/assignments/submissions/:id/review	Teacher review and scoring
GET /api/assignments/course/:courseId	Assignments by course
POST /api/assignments	Create assignment
GET /api/assignments/:id	Assignment detail
DELETE /api/assignments/:id	Delete assignment
PATCH /api/assignments/:id/publish	Publish assignment
POST /api/assignments/:id/submit	Student submission
GET /api/assignments/:id/submissions	All submissions
GET /api/assignments/:id/submission/:sid	Student submission detail

8.6 Materials

Endpoint	Description
POST /api/materials/upload	Upload file
POST /api/materials/link	Add external link
GET /api/materials/download/:id	Download file
DELETE /api/materials/:id	Delete material

8.7 Notifications

Endpoint	Description
GET /api/notifications/:userId/:role	Fetch notifications
PUT /api/notifications/read/:notifId	Mark one as read
PUT /api/notifications/readall/:userId/:role	Mark all as read

8.8 Buckets

Endpoint	Description
POST /api/buckets/threshold/:courseId	Set promotion threshold
GET /api/buckets/student/:sid/course/:cid	Student bucket
PUT /api/buckets/student/:sid/course/:cid	Update bucket
GET /api/buckets/course/:courseId/overview	Class distribution
POST /api/buckets/promote	Promote student
GET /api/buckets/performance/:studentId	Performance summary

9. Database Schema

9.1 Identity Collections

Collection	Key Fields
Student	name, email (unique), studentId, department, semester, status, enrolled-Courses[], gpa, level
Teacher	name, email (unique), employeeId, status, assignedCourses[]
Admin	name, email, adminType, permissions[], approvedStudents[], approvedTeachers[]
User	Legacy generic model; still referenced by some modules

Table 7: Identity Collections

9.2 Academic Core Collections

Collection	Key Fields
Course	courseCode (unique), courseName, department, semester, teacher (ref), enrolledStudents[]
Quiz	title, courseId, questions[], timeLimit, negativeMarking, difficulty, published
QuizResult	quizId, studentId, answers[], score, percentage, submittedAt
Assignment	title, courseId, questions[], modes, published
AssignmentSubmission	assignmentId, studentId, responses[], score, feedback, reviewedBy

Table 8: Academic Core Collections

9.3 Operations Collections

Collection	Key Fields
Material	title, type (file/link), url, courseId, uploadedBy, published, download-Count
Notification	title, message, targetRole, targetUser, readBy[], isActive

Table 9: Operations Collections

9.4 Progression Collections

Collection	Key Fields
SubjectThreshold	courseId, easyToMedium (score), mediumToHard (score)
StudentBucket	studentId, courseId, bucket (Easy/Medium/Hard), promotion-History[]

Table 10: Progression Collections

10. Frontend Component Architecture

10.1 Role-Based Layout Components

Component	Responsibility
StudentLayout	Sidebar: Home, Courses, Assignments, Materials, Quizzes, My Progress. AI Assistant nav item removed in this release.
TeacherLayout	Sidebar: Dashboard, Courses, Marks, Materials, Assignments, Quizzes, Grading, Analytics, Notifications.
AdminLayout	Sidebar: student/teacher management, course tools, assessments, and analytics.

Table 11: Layout Components

10.2 Student-Facing Components

Component	Responsibility
StudentPortal	Home dashboard: profile card with AI Insights button, quiz star summary, charts, weak areas, enrolled courses.
AILearningAssistant	Fetches data, calls Gemini backend, renders insights modal.
StudentQuizList	Lists available quizzes per course.
StudentQuizTake	Timed quiz interface with negative marking support.
StudentBucketProgress	Visual bucket-stage overview per subject.

Table 12: Student Components

10.3 Key Teacher Components

Component	Responsibility
TeacherQuizManagement	Create, edit, publish quizzes; Gemini question generation.
TeacherQuizMonitor	Live monitoring of ongoing quiz attempts.
SubjectMarks	Per-course enriched marks table.
BucketDashboard	Set thresholds; view class bucket distribution.
TeacherNotifications	Role-filtered notification feed with read tracking.

Table 13: Teacher Components

11. Security and Data Integrity

11.1 Authentication and Authorisation

- All protected routes require a valid JWT verified by Express middleware before any controller logic executes.
- Role-appropriate routes are enforced by ProtectedRoute components and the dynamic router factory on the frontend.
- Teacher and student route trees are fully separate, preventing cross-role data access.

11.2 Data Integrity Mechanisms

- Unique courseCode index on Course prevents duplicates.
- Soft deletes on notifications use the isActive flag.
- Approval gates prevent students and teachers from accessing platform features until explicitly approved by an Admin.

11.3 AI Safety and Fairness

- The Gemini Insights module processes only data that exists in the system – it cannot invent scores or subjects.
- GEMINI_API_KEY is stored exclusively in server/.env and is never transmitted to the frontend.
- AI-generated questions are surfaced to teachers for review before publication and are never auto-published.
- Faculty retain full manual override capability for bucket assignments.

12. Current Strengths and Limitations

12.1 Strengths

- 1. Comprehensive domain coverage** across auth, courses, quizzes, assignments, materials, notifications, buckets, and AI insights.
- 2. Clean extensibility** through separate models, REST grouping, and modular React components.
- 3. Practical AI integration** – the Gemini panel uses real student data, embedded directly in the natural dashboard workflow.
- 4. Multi-source marks aggregation** combining quiz and manual marks into unified subject-level reporting.
- 5. Adaptive difficulty tracking** via buckets provides lightweight personalisation without a full recommendation engine.
- 6. Production-oriented patterns:** JWT guards, soft-delete, file upload handling, and environment-variable-based secret management.

12.2 Limitations and Improvement Opportunities

- 1.** Legacy User model coexists with role-specific models and should be consolidated.
- 2.** No centralised validation middleware; input checks are inconsistent across controllers.
- 3.** Automated test coverage is minimal; critical workflows lack integration tests.
- 4.** No audit log for sensitive admin operations.
- 5.** Pagination is absent from several list endpoints.
- 6.** Attendance and timetable modules are not yet implemented (see Section 13).

13. Future Scope

13.1 Short-Term – Planned Modules

- **Attendance Management** – mark and report attendance per class session with a unique compound index on (date, subject, teacherId) to prevent duplicates. Will include teacher-wise and student-wise reports.
- **Timetable Module** – create weekly timetable slots with automatic duplicate and time-conflict detection, teacher-specific views, and soft-delete.
- API versioning (/api/v1/...) and consistent error contracts.
- Centralised RBAC middleware guard factory.
- Event-driven notifications (auto-notify on quiz publish, assignment grade, etc.).

13.2 Medium-Term

- **AI study planner** – generate a personalised weekly study schedule from weak areas and deadlines using Gemini.
- Automated assignment rubric grading with per-criterion AI feedback.
- Proctoring enhancements: tab-switch detection and time-per-question analysis during quizzes.
- Parent/guardian read-only portal for attendance and progress reports.
- Exportable reports (PDF/CSV) and semester-over-semester trend charts.

13.3 Long-Term

- **Multi-institution SaaS tenancy** with isolated data namespacing.
- Native mobile applications (React Native) for students and faculty.
- **Predictive analytics** – ML-based early dropout risk detection and targeted intervention planning.
- Integration with external ERP/SIS and national examination systems.

14. Risk Register

Risk	Impact	Mitigation
Legacy model inconsistency	Stale or conflicting student records	Unify User model; write migration scripts
Security vulnerabilities	Unauthorised data access	JWT guards on all routes; input sanitisation
Scale at large cohort size	Slow queries, timeouts	Add pagination and compound indexes
AI hallucination	Incorrect questions or feedback	Human review gate; data-grounded prompts
API key exposure	Unauthorised Gemini usage	Server-side calls only; keys in .env
Data loss	Irreversible deletion	MongoDB Atlas backups; soft-delete strategy

Table 14: Risk Register with Mitigations

15. Conclusion

This project has successfully evolved from a basic role-based prototype into a **comprehensive, modular academic platform** with strong backend coverage, an extensible evaluation engine, and a meaningful AI integration that delivers real value to students.

The **AI Performance Insights module** – powered by Google Gemini – demonstrates how large language model capabilities can be embedded meaningfully into educational workflows. By grounding every API call in the student’s actual quiz results and assignment data, the module produces specific, credible, and actionable feedback rather than generic advice. The server-side service pattern in `geminiService.js` keeps the API key secure and decouples AI logic from route controllers, making future model upgrades a one-line change.

The system is production-ready in its core modules and establishes a solid architectural foundation for the short-term roadmap (attendance, timetable, API versioning) and long-term goals (adaptive AI study planning, predictive dropout analytics, mobile clients, and multi-institution SaaS deployment).

Sarthak Pandit
2023BTECH072

Tushar Parihar
2023BTECH093

16. References

1. Pujiati, D., & Averina, M. *International Journal of Multidisciplinary: Applied Business and Education Research*.
2. Zhang, X., et al. (2024). An Effective Learning Management System for Revealing Student Performance Attributes. *2024 International Conference on CITS*, IEEE.
3. Guo, et al. (2022). Predictive Analytics for At-Risk Students using Machine Learning on LMS data. Brightspace LMS Study.
4. MongoDB Documentation. <https://www.mongodb.com/docs/>
5. Express.js Documentation. <https://expressjs.com/>
6. React Documentation. <https://react.dev/>
7. Node.js Documentation. <https://nodejs.org/en/docs>
8. Tailwind CSS. <https://tailwindcss.com/docs>

Appendix A – Data Model Quick Reference

Identity Student, Teacher, Admin, User (legacy)

Academic Course, Quiz, QuizResult, Assignment, AssignmentSubmission

Operations Material, Notification

Progression SubjectThreshold, StudentBucket

Planned Attendance, TimetableSlot

Appendix B – Environment Variables

```
#           Mongo   DB
MONGO_URI= mongodb://localhost:27017/lms_db

# JWT
JWT_SECRET=your_jwt_secret_key
JWT_EXPIRES_IN=7d

#   Google   Gemini   AI
GEMINI_API_KEY = your_google_gemini_api_key

# Server
PORT=5000
NODE_ENV = development
```

Listing 8: server/.env.example

Appendix C – Setup & Run Instructions

```
# 1. Clone repository
git clone https://github.com/sarthakpdt/LMS-System-Minor-Project.git
cd LMS-System-Minor-Project

# 2. Backend
cd server
npm install
cp .env.example .env           # fill in GEMINI_API_KEY, MONGO_URI, JWT_SECRET
npm run dev                    # http://localhost:5000

# 3. Frontend (new terminal)
cd ../client
npm install
npm run dev                    # http://localhost:5173
```

Listing 9: Development setup